

TCS GenAI Lab — Verified Capabilities

Probed live 2026-07-26 against `https://genailab.tcs.in/v1` . Every row below was measured, not inferred from the catalog. `models.json` advertises **30** models; **7 are dead**. Do not trust `/v1/models` as an availability list.

1 · What actually works

Model ID	Latency ¹	tok/s ²	Max input ³	JSON	Tools	Verdict
genailab-maas-gpt-5.4	1.3s	91	≥900k			Workhorse. Whole-repo context.
genailab-maas-gpt-5.4-mini	1.1s	108	250k–500k			Fastest good model. Bulk codegen.
genailab-maas-gpt-5.4-nano	1.4s	—	250k–500k			Cheap classify/extract
genailab-maas-gpt-5.2	2.1s	—	—			Works
genailab-maas-gpt-5.1	1.4s	77	—			Works
genailab-maas-gpt-5.0	2.0s	—	—			Reasoning tokens burned on trivial prompts
azure/genailab-maas-gpt-5-mini	1.6s	—	—			Works
azure/genailab-maas-gpt-4.1	1.1s	94	100k then 429			⚠ Rate-limits on big context
azure/genailab-maas-gpt-4.1-mini	1.2s	60	—			Works
azure/genailab-maas-gpt-4.1-nano	1.1s	—	—			Works
genailab-maas-gpt-4o	1.1s	—	128k			Slow at length (12.9s @100k)
azure/genailab-maas-gpt-4o-mini	1.1s	—	128k			Safe default, small jobs only
genailab-maas-gpt-35-turbo	1.2s	—	—			Legacy
gemini-3-flash-preview	1.4s	79	≥900k			⚠ Erratic long-context recall
gemini-3.1-pro-preview	3.3s	100	≥900k			⚠ Erratic long-context recall
gemini-2.5-pro	1.3s	124	≥500k	✗ raw		⚠ 1918 reasoning tok for 337 chars
gemini-2.5-flash	1.4s	—	—			Works
gemini-2.5-flash-lite	1.3s	—	—	✗ raw		Works
azure_ai/genailab-maas-DeepSeek-R1	1.9s	—	—	✗	✗ 400	Leaks <think> ; no tool calling
azure/genailab-maas-text-embedding-3-large	—	—	—	—	—	3072-dim , working

¹ trivial prompt, warm. ² measured on a real codegen task. ³ needle-in-haystack, verified recall.

2 · Dead — do not reference

Model ID	Failure
genailab-maas-gpt-5.3-codex	404 / 400 / 429 — never once succeeded
genailab-maas-gpt-5.2-codex	Succeeded once, then 404×2, 429×4 — unusable
azure_ai/genailab-maas-Llama-3.2-90B-Vision-Instruct	404 DeploymentNotFound
azure_ai/genailab-maas-Llama-3.3-70B-Instruct	404 DeploymentNotFound
azure_ai/genailab-maas-Llama-4-Maverick-17B-128E-Instruct-FP8	404 DeploymentNotFound
azure_ai/genailab-maas-Phi-4-reasoning	404 DeploymentNotFound
genailab-maas-DeepSeek-V3-0324	410 model_deprecated
gemini-2.0-flash-001	404 Publisher model not found

The two codex models are the trap. Their names promise the best coding model on the gateway and they are the least reliable thing on it. Anyone who defaults `LLM_MODEL` to a codex ID loses the morning.

3 · Findings that change how you build

gpt-5.4 swallowed 900,000 tokens and recalled the needle exactly, in 19.5s. That is the single most valuable fact here. It means an entire hackathon repo fits in one call — no chunking, no RAG, no vector DB for code-comprehension work.

Concurrency is free; total tokens are not. 16 simultaneous requests to the same model returned `16 ok / 0×429` for all five models tested, and an 8-model mixed fan-out completed 8/8 in **2.8s wall**. Every 429 observed came from *large-context* calls (250k+). The limiter is tokens-per-minute, not requests-per-minute:

- Small requests → parallelize hard (16-way is safe).
- 250k+ requests → serialize, one at a time.

Gemini long-context recall is unreliable. On a degenerate haystack, `gemini-3-flash` failed recall at 100k/250k/500k but passed at 900k; `gemini-3.1-pro` failed at every size. Real prose/code will do better than this adversarial filler, but do not put Gemini on a needle-retrieval path without testing your own data. The GPT-5.4 line recalled correctly at every size.

Two capability gaps: DeepSeek-R1 rejects `tools` with a 400 and leaks raw `<think>` blocks into `content`. Gemini `2.5-pro / 2.5-flash-lite` won't emit bare JSON without fences even when instructed.

Correction to `models.md`: its "Known mismatch" note is resolved — `genailab-maas-gpt-4o` (no prefix) is correct and live. `azure/genailab-maas-gpt-4o` is not in the catalog and does not resolve. `CLAUDE.md` §6 in `hack2-master` lists that wrong ID plus `gemini-3-pro-preview`, which is also not a real ID (the live one is `gemini-3.1-pro-preview`).

4 · Reproduce

```
python probe.py    # availability, JSON, tool-calling across all 30  
python probe3.py   # needle-in-haystack at 100k/250k/500k/900k  
python probe4.py   # concurrency burst + mixed fan-out
```